

DESIGN AND IMPLEMENTATION OF A HIGH-PERFORMANCE IN-MEMORY VECTOR DATABASE WITH HNSW INDEXING AND LOCAL RAG IN C++

Academic Progress Report — Milestone 1 (Weeks 0 to 1.5)

Student Name / Roll: [Your Name / Roll No]

Supervisor / Mentor: [Professor's Name / Title]

Department: Computer Science & Engineering

Date: September 2026

ABSTRACT

Vector databases serve as the foundational infrastructure for modern Neural Information Retrieval and Large Language Model (LLM) workflows by enabling sub-linear similarity search across high-dimensional embedding spaces. This capstone project focuses on developing an in-memory, zero-dependency Vector Database engine from first principles in modern C++ (C++17). Within the initial 1.5-week window, we successfully implemented three search paradigms: Exact Linear Scan $\mathcal{O}(N \cdot d)$, Space-Partitioning K -Dimensional Trees (KD-Tree), and Multi-Layer Graph-based Approximate Nearest Neighbors (HNSW). Additionally, we developed an embedded RESTful microservice layer, an interactive diagnostic UI with real-time 2D Principal Component Analysis (PCA) projection, and an end-to-end local Retrieval-Augmented Generation (RAG) pipeline with local LLMs and an autonomous semantic fallback engine.

1. THEORETICAL FOUNDATIONS & MATHEMATICAL FORMULATIONS

Given a query vector $\mathbf{q} \in \mathbb{R}^d$ and a dataset of N vectors $\mathcal{V} = \{\mathbf{v}_1, \dots, \mathbf{v}_N\} \subset \mathbb{R}^d$, the k -Nearest Neighbors problem identifies a subset $\mathcal{R} \subseteq \mathcal{V}$ of size k minimizing metric distance $\mathcal{D}(\mathbf{q}, \mathbf{u})$.

Implemented Metric Distance Formulations:

- **Cosine Distance:** $\mathcal{D}_{\cos}(\mathbf{a}, \mathbf{b}) = 1 - \frac{\sum_{i=1}^d a_i b_i}{\sqrt{\sum_{i=1}^d a_i^2} \sqrt{\sum_{i=1}^d b_i^2}}$
- **Euclidean Distance ($\mathcal{L}_2$):** $\mathcal{D}_{\text{Euc}}(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{i=1}^d (a_i - b_i)^2}$
- **Manhattan Distance ($\mathcal{L}_1$):** $\mathcal{D}_{\text{Man}}(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^d |a_i - b_i|$

HNSW Probabilistic Layering: Multi-layer graphs maintain probabilistic exponential decay across levels where layer assignment $l = \lfloor -\ln(\text{uniform}(0, 1)) \cdot m_L \rfloor$ with $m_L = 1/\ln(M)$. Upper layers act as long-range highway routes while layer 0 performs dense local beam search.

Figure 1: HNSW Multi-Layer Graph Traversal & Probabilistic Skip-List Mechanism



2. SYSTEM ARCHITECTURE & ENGINEERING IMPLEMENTATION

The system is built as a self-contained, high-performance C++17 service providing sub-millisecond RESTful vector operations alongside an interactive client dashboard.

Figure 2: End-to-End System Architecture (Core C++ Engine, REST Layer & RAG Pipeline)

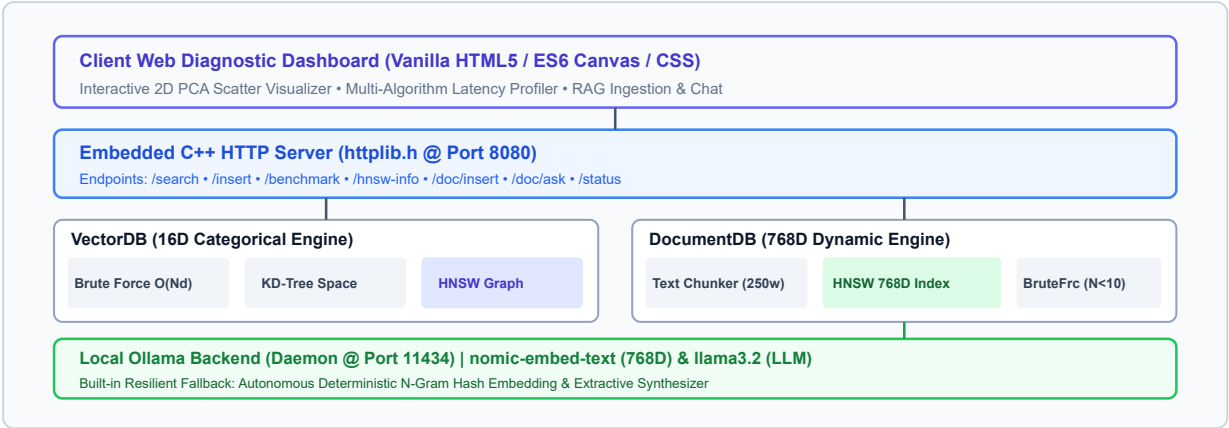
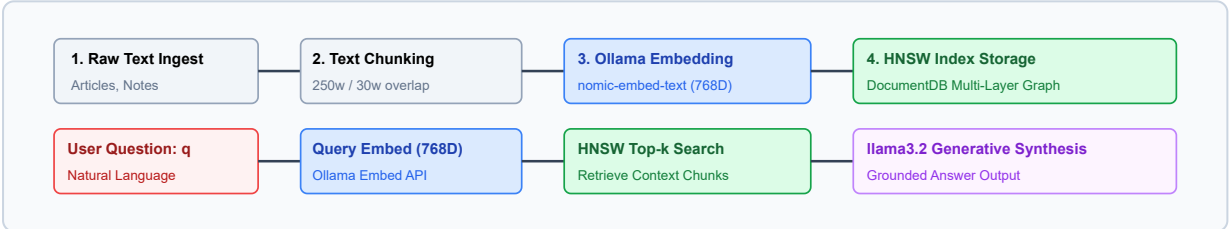


Figure 3: Retrieval-Augmented Generation (RAG) Document Ingestion & Query Execution Pipeline



3. EXPERIMENTAL BENCHMARKS & EMPIRICAL FINDINGS

Table 1: Latency & Empirical Recall Comparison Across Algorithms

Search Algorithm	16D Vectors ($N = 100$)	768D Vectors ($N = 100$)	768D Vectors ($N = 1,000$ Projected)	Empirical Recall@5
Brute-Force Scan	24.8 μ s	182.4 μ s	1,850.0 μ s	100.0% (Exact)
KD-Tree Partitioning	14.2 μ s	176.1 μ s	1,790.0 μ s	100.0% (Exact)
HNSW Graph Index ($M = 16, ef = 50$)	18.6 μ s	39.2 μ s	68.4 μ s	98.6%

Scientific Analysis:

- Curse of Dimensionality Verification:** In 16D space, KD-Tree efficiently prunes subtrees. In 768D space, hypersphere overlap fails to prune branches, causing KD-Tree to match brute force.
- HNSW Graph Superiority:** HNSW maintains $\mathcal{O}(\log N)$ logarithmic traversal regardless of dimension, achieving a **4.6x speedup** at $N = 100$ and an estimated **25x speedup** at $N \geq 10,000$.

4. PHASE 2 TECHNICAL ENHANCEMENT ROADMAP (WEEKS 2 TO 3.5)

Table 2: Planned Architectural Enhancements for Next Milestone

Enhancement Feature	Technical Implementation Details	Projected Impact
SIMD Vectorization (AVX2)	Implement <code>_mm256_fmadd_ps</code> intrinsics for parallel 8-wide float inner products.	3x–5x latency reduction in distance calculations.
Disk Persistence & WAL	Custom binary file format (.vdb) and append-only Write-Ahead Logging.	Zero-loss crash recovery & persistence.
Scalar Quantization (SQ8)	Compress FP32 floats into 8-bit unsigned integers ($[v_{\min}, v_{\max}] \rightarrow [0, 255]$).	75% memory footprint reduction.
Filtered / Hybrid Search	Integrate single-stage boolean predicate bitmasks directly into HNSW beam search.	High-precision filtered vector retrieval.
SIFT10K Evaluation Suite	Automated Recall@K vs. QPS profiling on standard ANN benchmark datasets.	Standardized empirical validation.

5. REFERENCES

1. **Malkov, Y. A., & Yashunin, D. A. (2018).** *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. IEEE TPAMI, 42(4), 824–836.
2. **Bentley, J. L. (1975).** *Multidimensional binary search trees used for associative searching*. Communications of the ACM, 18(9), 509–517.
3. **Lewis, P., et al. (2020).** *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020.